



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068240216

姓 名： 李晓翠

实验

【实验目的】

1. 掌握 LLM API 的基础调用方法，实现支持流式输出和历史上下文记忆的交互式终端聊天程序。
2. 理解并实践 LLM 的 Function Calling（工具调用）机制，使模型获得访问模型外知识的能力。
3. 基于 Python 标准库开发文件系统操作工具函数，实现 AI 智能体对本地文件的管理能力。
4. 掌握 Git 版本控制基础操作，包括仓库初始化、变更提交和代码版本管理。

【实验内容】

1.2 实验过程：

1. Git 仓库初始化与版本管理：

安装 Git 软件，配置全局用户名和邮箱，在开发平台（Cursor）的源代码管理面板中点击"初始化仓库"，生成 .git 目录。编写代码后，在面板中查看变更（Changes），点击"+"暂存文件，输入提交信息，完成首次提交（Initial Commit）。



2. 设计第二个教学文档，支持终端界面输入聊天内容、支持流式输出、支持历史聊天记录自动添加到上下文，直到用户 `ctrl+c` 退出终端，否则一直循环。更新 README 文件。
3. 新建 `practice02` 目录。学习新增工具调用功能。需要开发 5 个 `function`：列出某个目录下有哪些文件（包括文件的基本属性、大小等信息）；修改某个目录下某个文件的名称；删除某个目录下的某个文件；在某个目录下新建 1 个文件，并且写入内容；读取某个目录下的某个文件的内容

4. 基于 practice01 的 chat_client.py 【根据自己的代码名称进行修改】
代码，在 practice02 目录下开发一个新的代码，把这五个工具调用的能力作为系统提示词发送给 LLM，实现工具调用

1.2 实验结果：



设计第二个教学文档，支持终端界面输入聊天内容、支持流式输出、支持历史聊天记录自动添加上下文，直到用户ctrl+c退出终端，否则一直循环。更新 README文件

Builder

> 思考过程

创建第二个教学文档，实现终端交互式聊天功能，支持流式输出和历史记录。

 llm_chat.py F:\人工提示\practice\practice01\l... +238 -0 [查看变更](#)

 4/4 已完成 ×

- ✓ 创建practice02目录
- ✓ 实现5个工具函数：列出文件、修改文件名、删除文件、新建文件、...
- ✓ 基于practice01代码开发具有工具调用能力的新代码
- ✓ 更新README文件

Thought

已完成practice02目录下的工具调用功能开发：

4/4 已完成

- ✓ 理解项目结构和现有代码
- ✓ 在tools.py中新增curl网络访问function
- ✓ 在tool_chat.py中集成新的curl function
- ✓ 更新README.md文档

Thought

已完成所有任务：

1. 项目理解

- 项目是一个AI智能体开发教学项目
- 包含两个练习目录：practice01（LLM基础调用）和practice02（工具调用）

2. 新增curl网络访问工具

You: 列出目录

Assistant:

```
```json
{
 "tool_call": {
 "name": "list_files",
 "parameters": {
 "directory": "/"
 }
 }
}
```

```
问题 输出 调试控制台 终端

=== LLM工具调用聊天终端 ===
可用工具：列出文件、修改文件名、删除文件、新建文件、读取文件
输入消息开始聊天，按Ctrl+C退出
=====
===

你：今天的日期是？

AI正在思考...
```

Git 版本管理验证：

仓库初始化成功，生成 .git 目录；首次提交记录：git log 显示提交信息；代码变更可追踪，文件状态正常

### 【实验总结】

本次实验完成了两个递进式模块的开发。在 practice01 基础聊天模块中，我掌握了 OpenAI 兼容 API 的调用方式，通过 OpenRouter 平台接入 Qwen3-Coder 模型，实现了流式响应技术以提升用户体验，同时设计了历史上下文管理机制，使程序能够支持多轮连贯对话，直到用户通过 Ctrl+C 主动退出。在 practice02 工具调用模块中，我深入理解了 LLM 的 Function Calling 机制，即模型通过分析用户意图自动选择并调用预定义工具，使用 JSON Schema 描述工具能力实现了 AI 与外部系统的标准化对接，并开发了五个文件系统操作工具，使 AI 具备了实际的本地文件管理能力。

关于工具调用机制的核心流程，用户的输入首先经过 LLM 进行意图识别，模型判断是否需要调用外部工具；若需要，则生成工具调用请

求并以 JSON 格式传递参数，由本地执行具体的工具函数，随后将执行结果返回给 LLM，最终由 LLM 结合工具输出生成完整的自然语言回答。在这一机制中，工具描述的清晰程度直接影响模型的调用决策，因此必须精心编写 `description` 字段；同时参数需通过 JSON Schema 严格定义类型和必填项，并且错误处理机制必须完善，以避免工具执行异常导致整个对话流程中断。

实验过程中也遇到了一些实际问题并逐一解决。例如模型有时不调用工具，经排查发现是工具描述不够清晰或用户意图表达不明确，通过优化 `description` 并增加示例说明后得到改善；参数传递错误多由 JSON 解析异常或类型不匹配引起，为此添加了参数校验和类型转换逻辑；中文文件名出现乱码是因为系统编码与 Python 默认编码不一致，通过显式指定 `encoding='utf-8'` 解决；流式输出存在卡顿现象，调整 `flush=True` 后确保了实时输出效果。

当前实验仍存在一定的局限性，例如仅实现了文件系统工具，应用场景相对有限。未来的改进方向包括增加网络访问工具以扩展信息获取能力，实现更复杂的工具链组合以支持多工具顺序调用，添加工具执行确认机制防止误操作，以及优化系统提示词以支持更自然的语言指令识别。

通过本次实践，我深刻理解了 LLM 作为“大脑”、工具作为“手脚”的 Agent 架构可行性。这种架构使 AI 不再局限于预训练知识，而是可以通过标准化接口与外部世界交互，这为大模型在实际业务场景中的落地应用奠定了重要基础。